

Schedule coordination through egalitarian recurrent multi-unit combinatorial auctions

Javier Murillo · Víctor Muñoz · Dídac Busquets ·
Beatriz López

Published online: 10 April 2009
© Springer Science+Business Media, LLC 2009

Abstract When selfish industries are competing for limited shared resources, they need to coordinate their activities to handle possible conflicting situations. Moreover, this coordination should not affect the activities already planned by the industries, since this could have negative effects on their performance. Although agents may have buffers that allow them to delay the use of resources, these are of a finite capacity, and therefore cannot be used indiscriminately. Thus, we are faced with the problem of coordinating schedules that have already been generated by the agents. To address this task, we propose to use a recurrent auction mechanism to mediate between the agents. Through this auction mechanism, the agents can express their interest in using the resources, thus helping the scheduler to find the best distribution. We also introduce a priority mechanism to add fairness to the coordination process. The proposed coordination mechanism has been applied to a waste water treatment system scenario, where different industries need to discharge their waste. We have simulated the behavior of the system, and the results show that using our coordination mechanism the waste water treatment plant can successfully treat most of the discharges, while the production activity of the industries is almost not affected by it.

Keywords Auction mechanisms · Schedule coordination · Egalitarianism

1 Introduction

Coordinating agents' schedules in environments with limited and shared resources is a key issue for letting the agents accomplish their goals, while ensuring that the resources are properly used (e.g. not overusing them). That is, the resources have to be distributed to the agents in such a way that all of them are able to perform their tasks. However, given that the resources are limited, this distribution cannot always match the agents' demands, since there may be conflicts between the tasks (e.g. more than one task requesting the same resource at the same time). In such cases, it would be impossible to satisfy all agents' requests, so some of them should be left without the needed resources. These situations may affect the performance of the agents, since they may fail to accomplish their goals (which could have economical impacts, for instance). Fortunately, in some domains, such as an industrial one, the agents (which may represent factories, production lines, etc.) may have some kind of buffer that allows them not to get stuck when a needed resource is not available. However, this situations should be minimized, since these buffers have a limited capacity and cannot be used indefinitely. Therefore, a coordination mechanism which takes this issue into account is needed. Such mechanism should then distribute the resources in such a way that the agents' schedules are modified the least possible. In this paper we propose the use of an auction-based mechanism that permits the coordination of the shared resources, while maintaining the agents' information private, including their buffers capacities. Particularly, we propose

J. Murillo · V. Muñoz · D. Busquets (✉) · B. López
Institut d'Informàtica i Aplicacions, Universitat de Girona,
Campus Montilivi, 17071 Girona, Spain
e-mail: didac.busquets@udg.edu

J. Murillo
e-mail: javier.murillo@udg.edu

V. Muñoz
e-mail: victor.munyo@udg.edu

B. López
e-mail: beatriz.lopez@udg.edu

to introduce the concept of fairness in the resource distribution process, so that the negative effects of the coordination are spread among all the agents, thus minimizing the impact in their original schedules.

Our research has been motivated by a real world problem, namely the *waste water treatment system*. In this domain, several industries perform waste discharges, which have to be treated by the treatment plant. The plant can be viewed as a set of limited resources, since its cleaning capacities are finite (it cannot treat all the incoming water if the contamination levels are too high, or if the flow cannot be absorbed). In case of overuse, the quality of the cleaned water diminishes, resulting in the contamination of the river basin. Thus, the industrial discharges should be coordinated so that all of them can be fully treated by the plant. Obviously, this coordination should not cause problems in the production processes of the industries, since this could cause production delays, missed delivery commitments, etc. that the industries do not want to afford. The benefits of using the coordination mechanism with egalitarianism in this real world problem are experimentally shown at the end of the paper.

The rest of the paper is organized as follows. In Sect. 2 we review some basic concepts regarding resource scheduling. Then we formalize the problem in Sect. 3, and detail the coordination mechanism in Sect. 4. We devote Sect. 5 to introduce the concept of fairness in the mechanism. We present the Waste Water Treatment domain in Sect. 6 and the experimental results in Sect. 7. Finally, we conclude the paper with some related work, discussion and future work.

2 Background

There is a huge work on scheduling since the beginning of Artificial Intelligence and Operational Research. In this section we briefly review the basic terms in these disciplines related to our problem, so that we focus the scope of the paper.

2.1 Resources

Regarding the resources, we can classify them according to the following features:

- *Durable vs perishable*: *Durable* resources are those that maintain their value as times goes by. That is, no matter when they are used, their properties are not changed. On the other hand, *perishable* resources are those that lose value when held over an extended period of time. In this case, these resources cannot be stored for a later use, but have to be immediately used. An example of a perishable resource is communication bandwidth, since it only has value if it is used, and it cannot be stored to later increase the bandwidth of a network connection.

- *Divisible vs indivisible*: This defines whether the resource can be indefinitely divided into as many units as desired (*divisible* resource) or it cannot be divided (*indivisible* resource). In this latter case, however, there may be multiple units of the resource (each of them being indivisible).
- *Controlled vs non-controlled*: Although usually the resources belong to an owner that can control the access to them, in some domains the resources are somehow “public”, meaning that anyone can use them, even without having been authorized to do so. This *uncontrolled access* makes the scheduling problem even harder. Examples of non-controlled resources can be found in natural resources, which are usually accessible to anyone, and for which there are no physical means of controlling this access.

2.2 Resource scheduling

Resource scheduling is concerned with finding an optimal sequence of resource use by a set of agents that need them in order to execute a set of tasks, which can have several constraints, such as precedence, due times, etc.

An issue that affects how the resource scheduling problem must be solved is whether the goal is to *generate* schedules for all the agents from scratch, or to *combine* existing schedules so that the resources are used in a proper way. In the former case, a schedule for each of the agents must be created, fulfilling the agents’ tasks constraints and also assuring that the resources are correctly used (for instance, not allowing more than one task to use a given resource at the same time). On the other hand, when combining schedules, each of the agents already has its schedule prepared, but it may be in conflict with the schedules of the other agents. Therefore, these schedules must be adapted in order to prevent such conflicts in the use of the resources. In this paper we deal with this latter issue.

Combining schedules could involve possible delays in using the resources, and the way this is handled by the agents determines the kind of combination required. That is, whether they can somehow continue with their work even if they lack some of the needed resources. This is the case of production chains, where products can be temporally stored in buffers before entering the next machine, so that the previous one can continue working. In this case, we can talk about an agent having a buffer that prevents it to get stuck until it gets the needed resources. Combining schedules in this scenario means then to use the buffers to delay some resource use, while keeping the agents’ activities alive.

2.3 Distributed scheduling

There are two main approaches for solving the scheduling problem (either generating or combining them). The

first one includes *classical scheduling* techniques, for which there is a vast literature [2, 26]. The main drawback of this approach is that the scheduler has all the power. That is, it makes all the decisions, and therefore does not let the agents take part in the generation of the schedules.

To allow for a more participative mechanism, a *distributed approach* can be taken. In this case, the agents can interact with the scheduler or between them in order to reach an agreement on the schedules. This approach can be divided into two main classes, depending on who is the responsible of decision making:

- *Centralized approach*: In this kind of scheduling, as in the classical one, there is a single entity that solves the problem. However, there is some interaction between the agents and the scheduler, so that the decisions are not totally taken by the latter. An example of such approach are market-based mechanisms, such as auctions [5, 25], where agents bid for using the resources and the auctioneer must decide which agents can do so. This paper is focused on using such kind of mechanisms, and we will describe them more deeply in the next section.
- *Decentralized approach*: In this approach there is no central entity, but the agents are themselves the ones solving the scheduling problem. They communicate with each other in order to reach a solution. This approach includes negotiation protocols, in which agents trade resources until they are all satisfied with the allocation [16], and also distributed constraint optimization problems [18, 23].

2.4 Auctions

As mentioned above, we have focused on the use of market-based mechanisms (and more concretely, auctions) to solve the scheduling problem. Auctions have been widely used in resource allocation problems with privacy issues [3], and we believe that they can also be applied to the schedule coordination problem.

In an auction process, there are the *items* to be auctioned, the *bidders* who wish to acquire the items, and the *auctioneer* who sells the items [14]. An auction starts with a *call for bids* by the auctioneer, announcing the items to be sold. Then, the bidders *submit their bids*, which are composed by the item(s) to buy and the price to pay for it (them). Once the auctioneer has gathered all the bids, it *clears the market* (i.e. decides which bids are the winners and the price the bidders must pay) and informs the bidders about the resolution. In a resource scheduling scenario, the items are the resources, the bidders are the agents willing to use these resources, and the auctioneer is an arbiter that mediates the possible conflicts among the agents.

Typically, the research on auctions is focused on the following three aspects (which can be tackled at the same time or independently):

- *Bidding policies*: define how each agent decides what to bid for and the value of the bids (that is, the price it is willing to pay for a given good or set of goods).
- *Market clearing* (or solving the *winner determination problem*): defines how the auctioneer selects the winning bids.
- *Pricing mechanism*: defines how the auctioneer decides the price to be paid by the winners.

One aspect that has not been deeply addressed in the field of auctions is the dynamicity of the environment. In most of the literature on auctions, these are assumed to be a “one-shot” problem; that is, solving an auction does not depend on previous auctions. However, in a resource scheduling scenario, where the resources might be continuously reassigned, there will be multiple auctions, one for each allocation step. This leads to what is known as *recurrent auction* [17], in which the same set of agents is participating in a sequence of auctions with the same goods being offered. This kind of auctions poses new problems, such as the following:

- *Resource waste problem*: this problem arises with perishable resources, since the auctioneer must sell them before they become useless. Therefore, it is faced with the trade-off of selling as much resources as possible, while still obtaining benefits by doing it.
- *Bidder drop problem*: in recurrent auctions, it may be the case that some agents are always losing, and some others are always winning. In such case, the losers may decide to leave the market-place, since they are not gaining any benefits. When this happens, the remaining agents can lower their prices, thus reducing the benefits of the auctioneer, or even causing the prices to collapse.
- *Asymmetric balance of negotiating power*: in most auctions, the prices for the goods mainly depend on the bids sent by the bidders (and specially of those sent by the richer agents). This means that the auctioneer has low power (or no power at all) for fixing the prices. In a recurrent auction, this may lead to the richer agents lowering the prices to unwanted levels by the auctioneer, thus collapsing the market.

In this paper we deal with the *market clearing* algorithm of recurrent auctions. Moreover, we have focused our mechanism to handle the *resource waste problem* as well as the *bidder drop problem*.

2.5 Social welfare

As mentioned before, the problem of resource scheduling is to find an optimal assignment of the resources to the agents. In this context, the optimality refers to the wellness of the allocation to the group of agents as a whole, not as individuals. This is what is known as *social welfare* [1], which is usually

a function of the utilities of each of the agents involved in the allocation problem.

Depending on how these utilities are combined, the system will behave differently. For instance, with an *utilitarian* social welfare (i.e. the social welfare is the sum of individual utilities), the goal of the system is to maximize the overall utility, while an *egalitarian* social welfare (i.e. defined as the utility of the agent that is worst off) introduces some degree of fairness in the system. Other social welfare functions and their corresponding system behavior are reviewed in [3].

3 Problem formalization

In the schedule coordination problem we are presented with a set of N agents, $\mathcal{A} = \{a_1, \dots, a_N\}$, sharing a set of M resources, $\mathcal{R} = \{r_1, \dots, r_M\}$, and a set of agents' schedules $\mathcal{S} = \{S_1, \dots, S_N\}$ that use the resources. Schedules can be modified, so that the use of the resource is delayed thanks to the presence of agents' buffers, $B_1, \dots, B_N$, locally managed by the agents. The problem then consists in finding the *best* coordinated schedules, given a cost function and subject to the constraints (i.e. no resource is overused) and preferences provided by the agents (as for example, their buffer capacity).

Our research is constrained to indivisible, perishable and uncontrolled resources. Then,

Definition 1 A resource R_i is characterized by a number of available units $\hat{Q}_i$.

Then, we note with $\hat{\mathbf{Q}} = (\hat{Q}_1, \dots, \hat{Q}_M)$ the vector containing the available units of each resource. As perishable resources, if some unit of any resource is not used at a given time t , it is lost. Since the access to the resources is not controlled, any agent can use a resource independently of the coordination agreement achieved among the agents. This is a non desirable situation that a coordination mechanism should avoid.

Definition 2 The *schedule* S_i of agent a_i is an ordered list of resources requests $S_i = \{req_i^1, \dots, req_i^{n_i}\}$ where,

- req_i^k is the k -th resource request of agent a_i ,
- n_i is the number of requests submitted by agent a_i (i.e. $n_i = |S_i|$).

Definition 3 A request req_i^k is defined by a bundle of resources $req_i^k = (\mathbf{Q}_i^k, s_i^k, d_i^k)$ where,

- $\mathbf{Q}_i^k = (q_{i1}^k, \dots, q_{iM}^k)$ is a vector containing the quantity of each resource agent a_i requires in its k -th request (i.e. $q_{ij}^k \in \mathbb{N}^+$ units of the j -th resource). If a request does not require any amount of the j -th resource, then $q_{ij}^k = 0$.

- s_i^k is the start time for the k -th request of agent a_i ,
- and d_i^k is the duration of the resources use in the k -th request of agent a_i .

Note that requests have a starting time and duration that cannot be changed, but the use of a buffer allows the effective use of the resource by the agent for a time. This time depends on the agent's buffer capacity, and the quantities of the delayed requests. Time is assumed to be discrete.

Definition 4 The *set of active requests at time t* , $\mathcal{AR}(t)$, is then defined as $\mathcal{AR}(t) = \{req_i^k \mid req_i^k \in S_i, i = 1..N \text{ (} t \in [s_i^k, s_i^k + d_i^k])\}$, where N is the number of agents.

For the sake of readability, we drop the i and k indices (agent and request indices respectively) and we re-enumerate the set of active requests as follows: $\mathcal{AR}(t) = \{req_1, \dots, req_{|\mathcal{AR}(t)|}\}$. So the i -th request in $\mathcal{AR}(t)$ is characterized by the bundle of resources, $\mathbf{Q}_i = (q_{i1}, \dots, q_{iM})$, the start time s_i , and duration d_i .

Definition 5 The j -th resource is a *conflicting resource at time t* if its demand in the set of active requests exceeds the resource capacity. That is:

$$|\mathcal{AR}(t)| \sum_{i=1} q_{ij} > \hat{Q}_j$$

Thus, let $\mathcal{RR}^c(t)$ be the set of indices of the conflicting resources at time t .

Definition 6 A *set of conflicting requests at time t* are the requests $\mathcal{AR}^c(t) \subseteq \mathcal{AR}(t)$ whose demand exceeds the resource availability of some of the resources. That is:

$$\mathcal{AR}^c(t) = \{req_i \mid req_i \in \mathcal{AR}(t) \text{ and } \exists j, q_{ij} \neq 0 \text{ and } j \in \mathcal{RR}^c(t)\}$$

With this constraint, each request is considered as a bundle. Therefore, no matter how many resources cause the conflict (it could be a single resource or more), the whole request is in conflict. That is, a request cannot be partially accepted (authorizing the use of only a subset of the resources); it is an "all or nothing" authorization. This modeling allows the agents to express the complementarity of the resources, i.e. the agent will only be satisfied if he gets all the resources in a bundle. In the case the agent wanted some non-related resources, it could send several requests, one for each of the resources.

We can analogously define, the set of active agents $\mathcal{AA}(t)$ as the set of agents requesting some resource at time t , and the set of conflicting agents $\mathcal{AA}^c(t)$.

Definition 7 A *coordinated schedule* is a schedule free of conflicting requests, that is,

$$\forall t \sum_{i=1}^{|\mathcal{AR}(t)|} q_{ij} \leq \widehat{Q}_j, \quad \forall j \in \{1..M\}$$

Agents are selfish, and have a private utility function that enables them to interact in order to decide upon a coordinated schedule.

Definition 8 Each agent a_i has a utility function $u_i : S \rightarrow \mathbb{R}$ that allows it to express to the other agents in the community about the utility of its schedule.

Let $\mathcal{U}$ be the set of all utility functions, $u_1, \dots, u_N$. Regarding the complete multi-agent system, an objective function $f : S \times \mathcal{U} \rightarrow \mathbb{R}$ can be defined, so the *best* coordinated schedule is the one that optimizes (maximizes or minimizes) it. Thus, we are faced with the problem of finding such best coordinated schedule. Choosing one or another f function would cause a different social welfare, as stated in the previous section. Since we are interested in treating equally all the agents, we focus on objective functions that provide egalitarian solutions.

3.1 Example

For instance, consider that we have three resources, $\{r_1, r_2, r_3\}$ with the maximum available units of 6, 5 and 6 respectively ($\widehat{\mathbf{Q}} = (6, 5, 6)$) and four different agents $\{a_1, a_2, a_3, a_4\}$ that request them according to the following data:

$$S_1 = \{req_1^1, \dots, req_1^{k_1}, \dots, req_1^{n_1}\}$$

$$S_2 = \{req_2^1, \dots, req_2^{k_2}, \dots, req_2^{n_2}\}$$

$$S_3 = \{req_3^1, \dots, req_3^{k_3}, \dots, req_3^{n_3}\}$$

$$S_4 = \{req_4^1, \dots, req_4^{k_4}, \dots, req_4^{n_4}\}$$

with,

$$req_1^{k_1} = (\mathbf{Q}_1^{k_1}, 0, 3)$$

$$req_2^{k_2} = (\mathbf{Q}_2^{k_2}, 2, 2)$$

$$req_3^{k_3} = (\mathbf{Q}_3^{k_3}, 0, 3)$$

$$req_4^{k_4} = (\mathbf{Q}_4^{k_4}, 3, 1)$$

and,

$$Q_1^{k_1} = (2, 5, 0)$$

$$Q_2^{k_2} = (2, 1, 3)$$

$$Q_3^{k_3} = (1, 0, 0)$$

$$Q_4^{k_4} = (5, 2, 0)$$

According to their starting times, the active requests at time $t = 0$ are $\mathcal{AR}(0) = \{req_1^{k_1}, req_3^{k_3}\}$. There is no conflict at this time ($\mathcal{AR}^c(0) = \emptyset$) since the resources required by these requests do not exceed the available units.

However, at time $t = 2$, we have $\mathcal{AR}(2) = \{req_1^{k_1}, req_2^{k_2}, req_3^{k_3}\}$ (see Fig. 1 top). Looking carefully at the resources we can see that there is a conflict on the use of the second resource at this time (there are 6 units requested when the available quantity is 5):

$$Q_1 = \begin{matrix} r_1, r_2, r_3 \\ (2, 5, 0) \end{matrix}$$

$$Q_2 = (2, 1, 3)$$

$$Q_3 = (1, 0, 0)$$

$$Total = (5, 6, 3)$$

$$\widehat{\mathbf{Q}} = (6, 5, 6)$$

Thus, although $req_3^{k_3}$ is also active at this time, it only requests the first resource, which is not in conflict, therefore it is not included in the conflicting request set, and thus $\mathcal{AR}^c(2) = \{req_1^{k_1}, req_2^{k_2}\}$.

4 Auction-based schedule coordination

Given a set of agents' schedules, composed by bundles of shared resources requests, schedule coordination consists on arriving to a compromise in the use of the resources so that none of them is overused and without changing the agents activity, thanks to the use of buffers. The resources can be used by several agents at the same time, while keeping the resources under their capacities.

To solve this scheduling problem, we could opt for a centralized approach (for example as in [2]), which given all the resources requests, generates a new schedule for each of the agents, without any conflict between simultaneous uses. Such a centralized approach implies that the central scheduler would make all the decisions. However, such decisions should be made by each of the agents, since they may not be willing to disclose private information related with their decision-making mechanisms. Thus, in order to preserve privacy, we have chosen to use a distributed approach for coordinating the schedules. Besides the agents competing for the resources, there is an additional agent that is in charge of performing this coordination (*coordinator agent*).

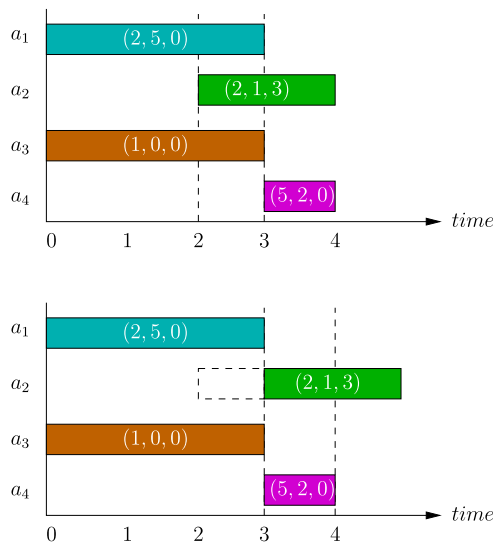


Fig. 1 Example of request conflicts. *Top*: conflict between agent a_1 and agent a_2 . *Bottom*: conflict between agent a_2 and agent a_4 after delaying the a_2 requests

4.1 Coordination mechanism

The mechanism proposed in this work is the following. A coordinator agent is set to deal with the scheduling coordination process. Then, all the agents inform the coordinator agent about the resources they want to use for a pre-established period of time (one week schedule, a fortnight, etc.). These different requests may be conflicting, since some agents may be interested in the same resources at the same time, and the number of units of each resource is limited. When such a conflict is detected, the agents involved are informed about it, and the coordination process (that is, the resource allocation) begins. The coordinator agent has to select a subset of the conflicting agents that will be authorized to use the resources, while some others will have to delay their resource usage. For this purpose, agents have a buffer that intermediates between the agent activity and the resource usage, so that loser agents are not forced to change their planned activity.

The resolution is done in a sequential way, treating one conflict at a time in chronological order. Once the current conflict is solved, the involved agents are informed about the resolution, and then each agent updates its resources use schedule. In case that an agent has not been authorized to use the resource at a given time, internally, the activity of the agent would not change, but instead of using the resources directly, it would use the buffer. That means that the agent will require the denied resources in a later period of time, in order to empty the buffer. For this purpose, the agent elaborates a new schedule reflecting such new requests, and informs the coordinator agent about it. This iterative process

Algorithm 1 Coordinator agent pseudo-code

```

1: receive schedules from agents
2: while not all requests authorized do
3:   for all time steps do
4:     if no conflict during current time then
5:       authorize current requests
6:     else
7:       solve conflict
8:       inform agents about resolution
9:       break for loop
10:    end if
11:  end for
12:  receive updated schedules from agents
13: end while

```

is performed until all requests are authorized by the coordinator agent according to the pseudo-code shown in Algorithm 1.

For instance, in the previous example, if agent a_2 loses the first auction, it would delay its requests until time $t = 3$ (see Fig. 1 bottom). At this time, the active requests are $\mathcal{AR}(3) = \{req_2^{k_2}, req_4^{k_4}\}$, all of them being in conflict (i.e. $\mathcal{AR}^c(3) = \mathcal{AR}(3)$), since there are only 6 units of the first resource and the total demand is for 7. So a new auction starts; as a solution of it, one of these conflicting requests would be delayed, and the coordinated schedule finally achieved.

We can see that the two main points of the coordination process are the *conflict detection* and *conflict resolution*. Regarding the latter, we propose the use of auctions. On the other hand, conflict detection has been formalized in the previous section: for each time t , the coordinator tests for conflicting resources $\mathcal{AR}^c(t)$; when one is found, the coordinator solves the problem by using an auction process. However, as auctions are recurrent, special attention should be paid in order to avoid possible cycles in solving the problems. That is, past auctioned solutions should not be revisited again. In the remaining of this section, we explain the details of this mechanism, mainly, the basic auction procedure, the bidding policy, the winner determination problem and the pricing mechanism.

4.2 Auction model

The goal of the auction is to select a subset of agents, which will be authorized to use the resources. The remaining agents should delay their resource use by using their buffers. The starting point of the conflict is the starting time of the request that first causes the restriction to be violated. The end time of the conflict is set to the time when the resources capacities constraint is met again. This end time is needed

by the losing agents, so that they know when is the earliest time they can try again to get the desired resources.

In our resource allocation process, the items are the resources, the bidders are the agents in conflict, and the auctioneer the coordinator agent. Since we have several resources and agents requests for bundles of them, we deal with *Combinatorial Auctions* [8]. Moreover, given that there are multiple units of each resource, it becomes a *Multi-Unit Combinatorial Auction* (MUCA).

The auctioneer agent calls for an auction in which the conflicting agents can place their bids to obtain the resources. So the participants in the auction are the agents in the agent conflicting set, $\mathcal{AA}^c(t)$.

However, to avoid cycles, the amount of resources to be auctioned varies depending on the solution of past auctions. That is, if agent a_i has participated in a previous auction at time t regarding a conflicting resource j involved in the req_i request, winning the resource at that time, and at a later time t' this request is still active, the agent request should not be involved in the auction at time t' . Otherwise, the outcome of auction at time t' could invalidate the outcome of auction at time t . Thus, the past auction should be repeated again, with the danger of causing a loop in the process. Figure 2 shows an example of such a situation. In solid line, the requests involved in the first auction are shown, while the dashed line encircles the requests involved in the second auction. Is it possible to see that the first request of agent 1 is common to both sets.

Let then $o(n)$ be the outcome of the n -th auction. Note that auctions are not performed at each time t but only when a conflict arises. So the time step is a unit, while the period between auctions can change.

For the sake of simplicity, the conflicting requests of $\mathcal{AR}^c(t)$ that motivates the n -th auction can be renamed as $req_1(n), req_2(n), \dots, req_{NC}(n)$, being $NC = |\mathcal{AR}^c(t)|$. Analogously, we can note by $\mathcal{AR}^c(n)$ the requests that participate in the n -th auction. Then, $conflictingAgent(req_i(n))$ refers to the agent that requires any request involved in the conflict, and $quantity(req_i(n), j)$ the units of the j -th resource in conflict.

Definition 9 An *auction outcome* $o(n)$ is characterized by the requests performed by all the agents in the conflict set $\mathcal{AA}^c(n)$ and a binary variable for each of them that specifies whether the request has been awarded or not. That is,

$$o(n) = \{(req_1(n), y_1(n)), (req_2(n), y_2(n)), \dots, (req_{NC}(n), y_{NC}(n))\}$$

Then, if $y_i(n) = 1$ it means that the $req_i(n)$ request was awarded. Conversely, if $y_i(n) = 0$, it means that it was not.

Thus, the requests of an auction should not contain requests already awarded in previous auctions. For this pur-

pose, we define the set of requests that are in common as follows.

Definition 10 The *set of common requests* $\mathcal{CR}(n)$ between auction n and the previous ones is defined as the requests awarded in any of the previous auctions that appear in the conflicting set of the next auction, $\mathcal{AR}^c(n)$. That is,

$$\begin{aligned} \mathcal{CR}(n) = \{ & req_i(n) | req_i(n) \in \mathcal{AR}^c(n) \text{ and } \exists n', i' \\ & (n' < n \text{ and } req_{i'}(n') \in \mathcal{AR}^c(n') \\ & \text{and } req_i(n) = req_{i'}(n') \\ & \text{and } y_{i'}(n') = 1) \} \end{aligned}$$

Then, we can define the reserved resource capacity according to the requests that cannot be auctioned again.

Definition 11 The *reserved resource capacity* $C_j(n)$ of the j -th resource for the n -th auction is the amount of units already assigned to agents in a previous auction. That is,

$$C_j(n) = \sum_{req_i(n) \in \mathcal{CR}(n)} quantity(req_i(n), j)$$

Thus, if $\widehat{Q}_j$ is the total amount of units of the j -th resource, according to the reserved resource capacity, the units that the auctioneer can sell in the n -th auction are the following:

$$\widehat{Q}_j(n) = \widehat{Q}_j - C_j(n)$$

Analogously, regarding agents, the ones that have won the request in a past auction would not participate in the next auction n with the same request. That is, if $conflictingAgents(\mathcal{CR}(n))$ are the agents that won in some previous auction a request common to the n -th auction, and $\mathcal{AA}^c(n)$ the conflicting agents of the n -th auction (so that $conflictingAgents(\mathcal{CR}(n)) \in \mathcal{AA}^c(n)$), then, automatically $conflictingAgents(\mathcal{CR}(n))$ are considered winners. The other way around, the agents that participate in the n -th auction are:

$$\widehat{\mathcal{AA}}^c(n) = \mathcal{AA}^c(n) \setminus conflictingAgents(\mathcal{CR}(n))$$

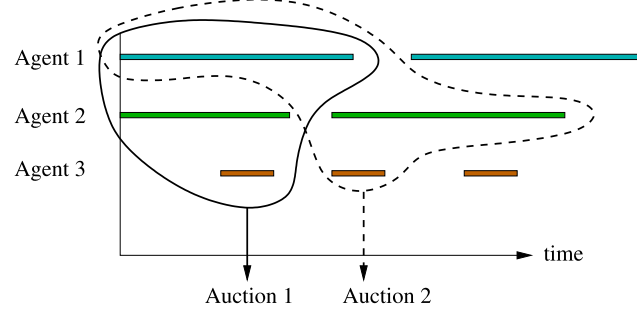
Then, these agents send a bid expressing the interest on their requests, and, once the auctioneer has received all the requests, it decides the next winners according to the winner determination algorithm.

4.3 Bidding policy

Each bid sent by the bidders has the following form:

$$b_i = \{req_i, v_i\}$$

Fig. 2 Example of request conflicts between two consecutive auctions



The first component of this tuple contains the characteristics of a request, as described in the previous section (resources quantities requested, start time and duration). Thus, we can rewrite a bid by explicitly showing the combination of resources embedded in a request as follows:

$$b_i = \{((q_{i1}, \dots, q_{iM}), s_i, d_i), v_i\}$$

so that combinatorial and multi-unit qualification of the auction is clearly stated.

The second component, $v_i \in \mathbb{R}^+$, is the value the corresponding bidder agent gives to the resources. A high bid indicates that the agent really needs (or wants) to use them, while a low bid indicates that the agent could wait before using the resources.

In our current model we have two assumptions regarding how agents decide upon their utility value. First, agents are honest. That means that they do not learn from past interactions in order to take advantage of the knowledge of the market. Second, agents are assumed to bid independently of their welfare. Thus, their bid value strictly depends on their internal information and necessities for achieving their goals. Observe, also, that the bidding policy is related to the utility functions described above: while the bid values express the interest on a single request of the scheduling, the utility does it regarding the overall agent's scheduling.

Note that each agent participating in the auction (that is, in $\widehat{\mathcal{AA}}^c(n)$) can send more than one bid, so the agent can express combinations that can differ from the original requests in conflict.

4.4 Winner determination problem

The goal of the auctioneer agent is then to select the winners that maximize a given objective function, subject to the constraints established by the resources capacities. Formally, to find the winners of the auction, the auctioneer agent must solve the following optimization problem, known as the *winner determination* or *auction clearing* problem, which, being equivalent to a multi-dimensional knapsack problem

[15], is NP-hard:

$$\begin{aligned} \max \quad & \sum_{i=1}^{NB} x_i \cdot g(b_i) \\ \text{s.t.} \quad & \sum_{i=1}^{NB} x_i \cdot q_{ij} \leq \widehat{Q}_j(n), \quad \forall j \in \{1..M\} \end{aligned} \quad (1)$$

where:

- NB is the total number of bids,
- $x_i \in \{0, 1\}$ represents whether bid b_i is assigned the resources ($x_i = 1$) or not ($x_i = 0$)
- $g(b_i)$ is the contribution of request i to the objective function to be maximized (minimized).
- q_{ij} is the quantity of resource j requested in bid b_i ,
- $\widehat{Q}_j(n)$ is the number of available units of resource j , taking into account the reservation issues of Definition 11.

There are different possibilities to model the g function, depending on the application domain. For example, g could be the value of the bid, or the value of some risk measure (if we have some information about the agents' trust and reputation), or a tradeoff of both, among others. Note, that g is at some extent related to the f function (see Definition 8). That is, g tries to express the interest of the auctioneer in a single auction, while f measures the wellness of the coordinated schedule after all the auctions have been completed.

If there is the free disposal condition, then the auctioneer can keep some of the units of the resources if the benefit is higher. However, this free disposal condition has to be minimized when dealing with perishable resources, as it can produce what is known as the resource waste problem. Since we are dealing with this kind of resources, we do not consider this condition, and therefore the auctioneer tries to allocate as many resources as possible.

For solving the winner determination problem presented in (1), we could use specific algorithms as the ones presented in [8] or some linear programming tool. For the experiments we have performed so far, the GNU Linear Programming toolkit [12] is fast enough. However, should the problem

size increase (due to more agents or more resources in the system), we could use more efficient algorithms or software (e.g. CPLEX, specialized CA algorithms). Moreover, we are currently working on the development of a solver for multi-unit combinatorial auctions that should better fit the characteristics of our problem [19]. Once it is completely finished, we will use it as the winner determination algorithm for our coordination mechanism.

4.5 Pricing mechanism

The coordinator does not gather any compensation for the use of the resources, since, as stated above, agents bid independently of their welfare. In some sense, this assumption skips the asymmetric balance problem that occurs in recurrent auctions. Although prices should be introduced in order to positively compensate agents that follow the coordination agreements (as well as penalties to the agents that do not), we leave this issues for future work. Thus, in the rest of the paper we will not deal with the problematic of the pricing mechanism.

5 Adding fairness

The auction process is repeated every time a new conflict is detected. This leads to a *Recurrent MUCA* model (RMUCA), where the same bidders are continuously competing for the same resources. One of the main concerns in recurrent auctions is to keep the agents interested in participating in the auction. If only a small subset of the agents wins the auctions, the rest of the agents may decide to leave the marketplace since they are not getting any benefit, in what is known as the *bidder drop problem* [17].

To avoid, or somehow decrease, the bidder drop problem, the recurrent auction process should have some degree of fairness, so that every agent has some possibility of winning from time to time. This would keep the agents attracted in taking part in the auction, which would benefit the global performance of the system. The inclusion of fairness can be somehow acting against optimality, since the result of an auction may differ from the optimal solution if a suboptimal solution is fairer. However, its long-term effect can have a better performance than a pure utilitarian view, since the duration of the recurrent auction may be longer with satisfied agents and the final outcome could be much higher. We have called this egalitarian approach to recurrent multi-unit combinatorial auction as *Egalitarian Recurrent MUCA* (or ERMUCA).

There are many ways in which fairness could be introduced into the auction. We have chosen to use a priority mechanism that takes into account the history of each agent in previous auctions. Since we are dealing with coordinated

but uncontrolled resources, priority helps the coordinator to model possible agents' behaviors that use the resource even when they are not authorized to do so. That is, although agents are honest, if they have their schedules continuously delayed, their buffers can become full and forced to use the uncontrolled resource. Thus, priorities try to capture this information related to the urgency to fulfill the agents necessities; or the other way around, they avoid having disobedient agents.

5.1 Priority model

Each agent a_i is assigned a priority value p_i in $[0, 1]$. A high priority value indicates that the agent should be authorized to use the resources. Conversely, a low priority value indicates that it would not be unfair to deny resource use to a given agent.

The priority values could be used by the clearing algorithm in very different ways: they could be transformed into new constraints to be satisfied by the solution, or directly designate the set of winning agents, among others. We have decided to use the priorities as a modifier of the bids sent by the agents.

More precisely, given a bid value v_i of an agent with priority p_i , a new bid value is computed as

$$v'_i = h(v_i, p_i) \quad (2)$$

This new value is the one used by the clearing algorithm to find the solution to the auction. The function h can also be designed in many ways, and it defines how much the fairness affects the auction solution. For example, when using the product $h(v_i, p_i) = v_i \cdot p_i$, fairness strongly affects the outcome of the auction.

5.2 Priority learning

Priorities are updated after each auction run and handled by the coordinator. They are related to the subjective probability that an agent disobeys due to their impossibility of accessing to the resources along the auction process.

Let $o(1), \dots, o(L)$ be the outcomes of the L auctions performed so far. We define a time window of length w corresponding to the last w observations, $\{o(L - w + 1), \dots, o(L)\}$. Let $\mathcal{O}^w$ be the set of all possible observations in a w length window frame. Then, priority is defined as a function $p : \mathcal{O}^w \times \mathcal{A} \rightarrow [0, 1]$. For example, p can be, among others:

1. the ratio between the number of times an agent has won an auction and the number of times an agent has participated in an auction
2. the ratio between the accumulated resources units an agent has won and the total resource units requested

3. the ratio between the diversity of the resources won by an agent and the diversity of the resources requested

Then, the priority p_i of an agent a_i is updated after each auction as follows:

$$p_i = p(\{o(L - w + 1), \dots, o(L)\}, a_i) \quad (3)$$

From (3), it can be seen that the priority of an agent only depends on the history of the auctions it has participated in, and is independent of what the other agents requested. Obviously, there is some indirect dependency between the agents, since the requests of all agents affect the outcome of the auction; however, when computing an agent's priority, only its requests and the outcomes of the auctions are taken into account.

In the following section we show different priority computation methods applied in the Waste Water Treatment domain.

6 Application: waste water treatment systems

Water is a vital natural resource, not only for urban and industrial consumption, but also as the main element to maintain any natural environment. Thus, the need of having a good treatment system is basic in order to account for the high demand it suffers. Moreover, the increase in population growth and industrial activity results in the harmful effect of having more contaminated waters, therefore making the treatment task harder.

Figure 3 depicts a typical water treatment system. The waste discharges from the industries are directed to the Waste Water Treatment Plant (WWTP) through the sewage system; these discharges are then treated in the plant, and the cleaned water is put into the river [13]. In order to ensure a proper treatment, two conditions must be met:

- Keep the *incoming water flow* below the WWTP hydraulic capacity (that is, the amount of water the plant can absorb at any instant in time); otherwise, the overflowed water goes to the river without receiving any treatment, increasing its contamination level.
- Keep the *contamination level* of the incoming water below the WWTP treatment capacity. Contamination is defined by a set of quality variables: oxygen demand, nitrogen level, etc. If the level of any of these variables is above the WWTP capacity, the water cannot be fully treated, increasing the contamination of the river. Moreover, if the levels were too high the whole treatment process could have to be stopped until these were regenerated. During this time, the plant could not accept any incoming water and it would be redirected to the river without any treatment.

Regulations regarding waste discharges already define the maximum amount of waste and the contamination level allowed to the industries. However, these are not sufficient to ensure the two conditions mentioned above. The problem is that regulations do not take into account that the WWTP thresholds can be exceeded when several industries discharge simultaneously. In such cases, none of the industries would be acting against the law, but the effect could cause overflow or overcontamination of the water arriving at the plant. Thus, it is easy to see that some coordination is needed in order to distribute the discharges so that they do not expose the plant to any dangerous situation. This coordination would improve the health of the river and its environment, and is directed towards a more integrated management of the river-basin. Being aware of this problem, the Laboratory of Chemical and Environmental Engineering (LEQUIA) of our university presented us the challenge of developing a mechanism to deal with these issues.

6.1 System modelling

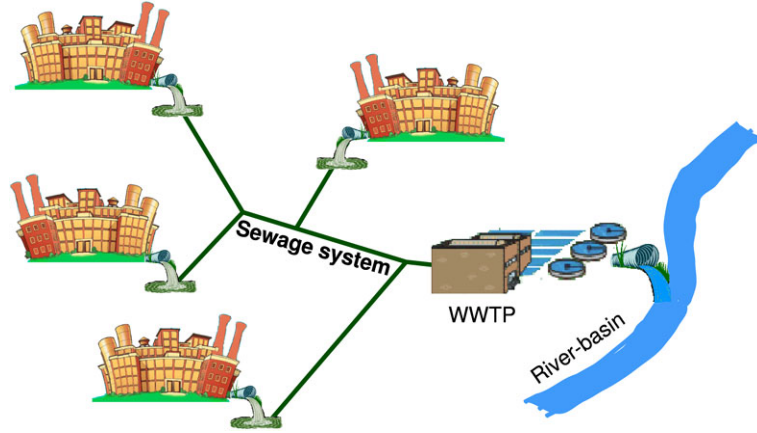
To reflect the physical separation between the different actors in this scenario (industries on one side, and the treatment plant on the other), we have modelled the Waste Water Treatment System as a multiagent system in which industries have been represented as bidding agents, and the WWTP as the auctioneer agent. The use of the multiagent system not only provides for greater privacy to each agent (since they only have to reveal minimal information), but it would also permit us to extend the system with other agent mechanisms when needed (such as learning, negotiation, etc.).

The shared resources are the hydraulic capacity of the WWTP and its contaminants levels (one resource per contaminant). For example, a given industry can require the following bundle of resources to perform one of its activities (COD is the Chemical Oxygen Demand, BOD is the Biological Oxygen Demand, TKN is the Total Kjeldhal Nitrogen and TSS is the Total Suspended Solids):

$$\begin{aligned} &(\{flow = 2250 \text{ m}^3/\text{day}, COD = 1500 \text{ g/m}^3, \\ &BOD = 500 \text{ g/m}^3, TSS = 750 \text{ g/m}^3, NKT = 40 \text{ g/m}^3\}, \\ &start = 10 \text{ h}, duration = 30') \end{aligned}$$

In order to apply our coordination mechanism, we assume that each industry has a tank (buffer) of a given capacity in which the unauthorized discharges can be temporarily stored. This assumption is plausible since currently most of the industries are already implementing retention tanks for different purposes.

To calculate the bid value of industry i for discharge d , v_{id} (recall that an agent may send more than one bid at the same auction), the industry agent a_i takes into account the

Fig. 3 Water treatment system

urgency for performing the discharge, based on the retention tank occupation of the industry. This urgency has been modeled according to the following expression:

$$v_{id} = \left[\frac{1}{1 + e^{-\alpha(2x_{id}-1)}} \cdot (1 - \min) \right] + \min \quad (4)$$

where $\min$ is the minimum value of a bid, α is a parameter indicating the function's slope and x_{id} is the percentage of occupation of the tank taking into account the actual discharge:

$$x_{id} = \frac{\text{tank occupation}_i + \text{discharge volume}_{id}}{\text{total tank capacity}_i}$$

In case an industry agent has its discharges denied, it tries to store the rejected discharge into the tank. However, if its tank is full, it disobeys and discharges anyway to the WWTP. The waste stored in the retention tank generates a new discharge request (or more if it is splitted), which will be treated as any other “normal” discharge. Moreover, the industries are allowed to perform two simultaneous discharges, one from the tank and one from production.

To calculate the priority p_i of agent a_i , the results of the past auctions are taken into account (see Sect. 5). In the current implementation we are using all of the auctions' outcomes, that is, the time window is set to L , the total number of auctions performed so far, and $O(L) = \{o(1), \dots, o(L)\}$ is the sequence of outcomes. Then, we have used two different methods for computing the priorities; the first one is based on the percentage of number of denied discharges (PD) and it is computed as follows:

$$p_i^{PD}(O(L), a_i) = \frac{\text{denied discharges}_i(O(L)) + 1}{\text{total requested discharges}_i(O(L)) + 2}$$

The second method is based on the percentage of volume denied (PV), and it is computed similarly:

$$p_i^{PV}(O(L), a_i) = \frac{\text{denied volume}_i(O(L)) + 1}{\text{total requested volume}_i(O(L)) + 2}$$

The initial priority of each agent is 0.5. Then, the priority is progressively and smoothly decreased as no conflicts arise and the agents have their discharges authorized. When an agent participates in an auction and its discharge is denied, its priority is increased.

The function chosen to modify the industry's bid according to its priority changes the value sent by the industry (see (2)). So, given v_{id} , the bid value by agent a_i for discharge d and its priority, p_i , the actual value used is $v'_{id} = h(v_{id}, p_i) = v_{id} \cdot p_i$.

In order to create more realistic scenarios, the behaviour of the industries have been simulated allowing them to disobey the WWTP decisions. This disobedience is modeled by a probability function that depends on the tank occupation. The probability that an industry disobeys is modelled as follows:

$$P = \left[\frac{1}{1 + e^{-\alpha(2x-1)}} \cdot (1 - \min) \right] + \min \quad (5)$$

where x is the tank occupation, α is a parameter indicating the function's slope, and $\min$ is the minimum probability value returned by the function. With such function, the probability to disobey increases as the tank occupation level is higher. This function is the same as the bid value, so when the bid value is high the probability of disobey is also high.

6.2 Implementation

To evaluate the proposed mechanism, a prototype of the system has been implemented using Repast [22], a free open source software framework for creating agent based simulations using the Java language. The simulation reproduces the process and the communication between the WWTP and the industries performing waste discharges.

7 Experimental results

We have tested the system in four different scenarios:

- In the first scenario there is no coordination among the industries.
- The second one uses the coordination mechanism and assumes that the industries always obey the WWTP decisions, as long as they have enough tank capacity.
- The third and fourth scenarios use coordination and the industries are allowed to disobey the WWTP decisions. In the third scenario the industries disobey the WWTP using (5) with $\min = 0$.
- In the fourth scenario the industries disobey the WWTP decisions with $\min = 0.1$.

The scenarios with coordination have been tested without priorities (RMUCA) and with priorities (ERMUCA), then we have used the two formulas for priorities, PD and PV .

In order to evaluate the system we have considered some quality measures based on different characteristics of the solution. Each of these characteristics, or a combination of them, could be used as the criteria to select which is the best schedule (the f function presented in Definition 8):

- number of overflows (NO) occurred during the execution of the schedules
- maximum flow overflowed (MFO), measured in m^3/day
- total volume overflowed (VO), in liters
- minimum percentage of authorized discharges (%MAD) among all agents
- standard deviation of percentage of authorized discharges (DAD)
- percentage of buffer discharges (%BD)
- standard deviation of percentage of buffer discharges (DBD)
- percentage of discharge denials being obeyed by the industries (%IO)
- percentage of discharge denials disobeyed due to the disobey function (%IDF)
- percentage of discharge denials disobeyed because the industries' tanks were full (%IDT)

The first three measures (NO, MFO and VO) reflect the overall performance of the system, from the point of view of the Treatment Plant. The lower these values, the more volume of waste water the plant has been able to successfully treat. The next four (%MAD, DAD, %BD and DBD) concern the fairness of the coordination mechanism. The minimum percentage of authorized discharges (%MAD) measures the worst case scenario an agent could face, so it can be seen as a lower bound on the performance any participating agent could have. On the other hand, the use of the buffer (%BD) measures how much the coordination mechanism has affected the original schedules of the agents. The deviations (DAD and DBD) measures how equally distributed are the negative effects of the coordination. Finally, the last three measures (%IO, %IDF and %IDT) reflect the obedience level of the agents when using the mechanism.

The experiments consisted of twenty simulations using a set of real data provided by LEQUIA of 4 industries during two weeks. The first one is a Pharmaceutical industry that increases its discharge flow during the week and does not perform any discharge during the weekend. The second one is a slaughterhouse that discharges a constant flow, except at the end of the day when it increases the discharge flow. The third one is a paper industry that discharges a constant flow during the seven days of the week. The fourth one is a textile industry, whose discharge flow oscillates during the day. In addition to these industries, we also consider the waste water coming from the city sewage. Although the flow coming from the city is not treated as an industry (since it cannot be stopped) and does not participate in the auctions, it affects the flow available for the industrial discharges. The hydraulic capacity of the WWTP is $32000 \text{ m}^3/\text{day}$ and we have selected the following constraints: a maximum BOD of 140 g/m^3 , a maximum COD of 900 g/m^3 , a maximum TKN of 85 g/m^3 and a maximum TSS of 430 g/m^3 .

Tables 1, 2 and 3 show the average and standard deviation of the twenty executions. The results show that with schedule coordination the number of overflows (NO), the maximum flow overflowed (MFO) and the volume overflowed (VO) are reduced (note one order of magnitude reduction in volume overflowed in Table 1). Moreover, with priorities the reduction is even higher.

Comparing the simulation with and without priorities the best results obtained in the second scenario with priorities are due to the reduction of the disobedience (%IDF and %IDT values) and the consequent increment of the %IO value (see Table 2). The priorities permit that the percentage of tank occupation be homogeneous. That is, if an industry has a discharge denied, it stores it in the tank. At the same time, its priority is increased so in a next auction round, it has more chances to get an authorization to empty the tank. If priorities are not used, the tank occupation could be continuously increasing, forcing the industry to disobey, as happens in the RMUCA scenario (see the %IO results on Table 2). Conversely, an industry that has always had its discharges authorized, has its tank empty, but its priority is low. Thus, it has few chances of winning a new authorization, and probably its tank occupation will increase. In the end, the tank level is homogeneous for all the agents.

Observing the results of scenario 2 (Obeying industries) in Table 3, we can see that the minimum percentage of authorized discharges (%MAD) obtained with priorities is higher than without priorities. Also the standard deviation of authorized discharges (DAD) is lower with priorities than without priorities. This means that the difference between agents is reduced with the use of priorities, increasing the fairness of the system and reducing the effects of the bidder drop problem. We can also see this effect in Fig. 4, which shows the percentage of authorized discharges for each industry, without priorities and with both types of priorities

Table 1 Experimental results (average and standard deviation)

		NO	MFO	VO
Obeying	w/o coord	80.00 (0.00)	9826.00 (0.00)	15213000.00 (0.00)
	RMUCA	24.00 (0.00)	4996.00 (0.00)	2917420.00 (0.00)
	ERMUCA PD	21.00 (0.00)	4996.00 (0.00)	2544730.00 (0.00)
	ERMUCA PV	22.00 (0.00)	4996.00 (0.00)	2604440.00 (0.00)
	RMUCA	54.30 (4.31)	14313.35 (1289.01)	9054222.55 (238358.66)
	ERMUCA PD	53.20 (3.11)	14502.90 (1353.67)	8961188.80 (139828.04)
	ERMUCA PV	53.15 (3.36)	13918.15 (1084.13)	9004427.90 (198402.56)
	RMUCA	104.35 (5.06)	13963.85 (1262.89)	11505287.05 (237799.94)
	ERMUCA PD	102.40 (6.57)	14037.70 (1729.73)	11495497.50 (403159.95)
	ERMUCA PV	102.45 (5.23)	13959.65 (1615.38)	11465226.30 (291274.80)
	RMUCA			
	ERMUCA PD			
	ERMUCA PV			

Table 2 Industries obedience results (average and standard deviation)

		%IO	%IDF	%IDT
Obeying	RMUCA	98.28 (0.00)	–	0.72 (0.00)
	ERMUCA PD	99.36 (0.00)	–	0.64 (0.00)
	ERMUCA PV	99.34 (0.00)	–	0.66 (0.00)
	RMUCA	99.08 (0.12)	0.92 (0.12)	0.00 (0.00)
	ERMUCA PD	99.05 (0.12)	0.95 (0.12)	0.00 (0.00)
	ERMUCA PV	99.05 (0.11)	0.95 (0.11)	0.00 (0.00)
Disobey min = 0	RMUCA	97.43 (0.23)	2.57 (0.23)	0.00 (0.00)
	ERMUCA PD	97.47 (0.26)	2.53 (0.26)	0.00 (0.00)
	ERMUCA PV	97.51 (0.15)	2.49 (0.15)	0.00 (0.00)
	RMUCA			
	ERMUCA PD			
	ERMUCA PV			

presented above. We can observe that when priorities are not used, the difference between the industries (which cor-

responds to DAD) are higher than when using priorities, while the average values are similar (64.20% without priori-

Table 3 Fairness results (average and standard deviation)

		%MAD	DAD	%BD	DBD
Obeying	RMUCA	59.45	3.46	15.83	3.36
		(0.00)	(0.00)	(0.00)	(0.00)
	ERMUCA PD	60.57	2.83	16.24	3.08
		(0.00)	(0.00)	(0.00)	(0.00)
	ERMUCA PV	60.94	2.41	16.44	3.51
		(0.00)	(0.00)	(0.00)	(0.00)
Disobey min = 0	RMUCA	62.35	6.57	13.18	2.10
		(0.42)	(0.38)	(0.29)	(0.27)
	ERMUCA PD	62.18	6.20	12.98	1.87
		(0.48)	(0.36)	(0.20)	(0.23)
	ERMUCA PV	62.24	6.55	13.23	2.14
		(0.49)	(0.34)	(0.32)	(0.33)
Disobey min = 0.1	RMUCA	65.85	7.05	12.38	0.81
		(0.43)	(0.26)	(0.31)	(0.34)
	ERMUCA PD	65.45	7.03	12.35	0.95
		(0.55)	(0.22)	(0.45)	(0.30)
	ERMUCA PV	65.56	7.24	12.47	0.79
		(0.53)	(0.27)	(0.38)	(0.36)

ties, 63.98% with priority PD and 64.24% with priority PV). Thus, when using them, the distribution of authorization is a bit fairer.

The %BD and DBO values show the utilization of buffers and represents the difference between the initial industries' schedules and the final schedules after coordination. It can be observed that when using priorities, these values are reduced, thus indicating that the impact on the industries' original schedules is minimized.

In the scenarios with disobedience the difference between using priorities or not using them is minimal due to the random behavior of the industries, which causes the priority values not being aligned with the real situation of each industry (i.e. an industry that has a discharge not authorized but disobeys the decision of the coordinator will empty its tank, however, its priority will be increased since it has been denied to discharge). This issue should be taken into account in the future when computing the priorities of the industries, thus better modelling the real situation of the system.

8 Related work

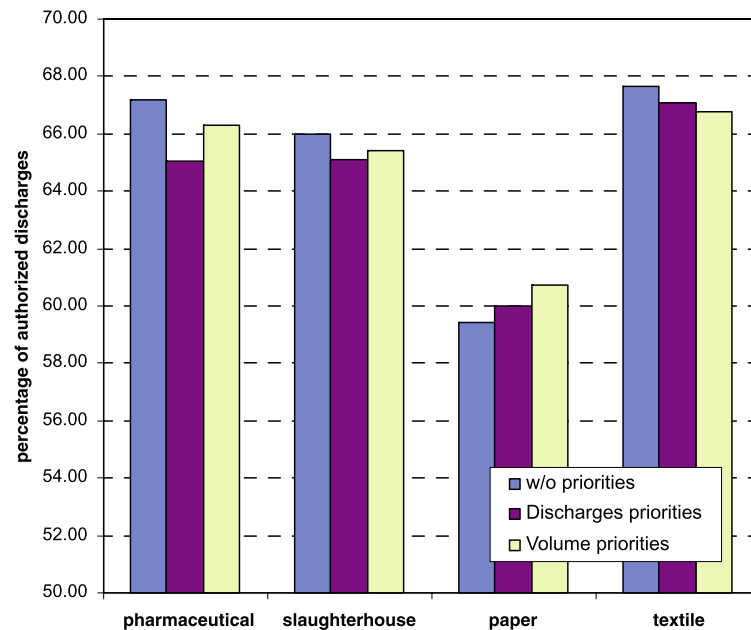
There are many approaches to handle schedule coordination: use a divide-and-conquer strategy [6, 24], solve it as a constraint optimization problem [7], or use auctions [9, 11], among others. We have followed the latter option, use a

market-approach to coordinate the schedules. The characteristics of our problem makes the auction to be continuously repeated, thus dealing with a recurrent auction. This kind of auction is recently being used for e-services markets, such as assigning advertising time in public displays [20] or in networking markets [17].

This latter work is closely related to our problem, since it tackles the bidder drop problem. We are also very interested in this problem, since we need to incentivize the agents to participate in the coordination process. In [17] the problem is solved by defining a more flexible winner determination algorithm, which takes into account the bidder's outcome history in past auctions. The goal of their work is to incentivize the bidders to stay in the market place, so that the prices do not collapse. We also use this history in order to compute the agents' priorities, however our objective is not economic but to obtain a fair distribution of the discharge authorizations. Regarding fair distribution of resources, it has also been studied in decentralized scenarios [4, 10]. However, in these works, the distribution is done through negotiation among the agents, and does not contemplate the use of auctions.

Finally, regarding work on waste water treatment systems, there has been much research on the internal treatment processes, but very little on coordinating the different systems involved. In [21] a negotiation approach to coordinate different WWTPs treating the same river basin is presented. However, the elements being coordinated in this work are the plants, leaving the industries aside.

Fig. 4 Distribution of authorized discharges among the industries



9 Conclusions and future work

In this paper we have presented a mechanism to coordinate schedules that have already been generated by selfish agents competing for a set of limited resources. Through the coordination, the individual schedules are adapted whenever a conflicting situation is detected. The new schedules contain a sequence of resources use that is distributed over time and decrease the risk of possible system failures (i.e. resources overuse). The core of the coordination mechanism is a multi-unit combinatorial auction (MUCA), in which agents can bid for the right to use a set of resources at a given time.

The auction is repeated each time a conflict arises, facing then a recurrent-MUCA model. In addition, we propose an egalitarian mechanism for the winner determination (ER-MUCA) in which a priority mechanism allows the introduction of fairness in the assignment of resources. With this mechanism, the assignments are evenly distributed among the agents. This is a very desirable property of a coordination mechanism, since it incentivizes agents to participate in it.

We have applied the mechanism to a waste water treatment system. In this scenario, different industries compete for performing waste discharges that require different resources of the plant. The results obtained through simulation show that the coordination mechanism achieves the goal of maintaining the resources below the WWTP thresholds. The results also show that the use of priorities provides a fairer solution of the auctions. Moreover, we have shown that these priorities add robustness to the solution, since even when the industries do not comply with the outcome of the coordination process, the overall system performance is not drastically affected.

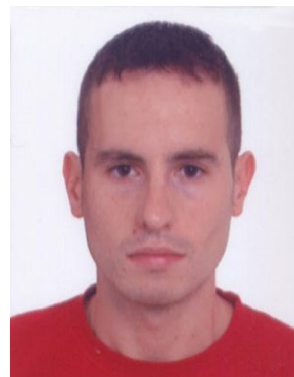
A strong assumption that we should relax is that agents bid honestly. The mechanism should be prepared to deal with strategic bidding (e.g. always bid the highest possible, bid low to lose and increase the priority, or any other the agent may come across). A possible solution could be the introduction in the mechanism of reservation prices (that is, the minimum price to which the auctioneer is willing to sell a resource). The idea is that the auctioneer does not have a single reservation price, as usual, but one for each bidder. These reservation prices would be updated after each auction and they would adapt to the behavior of each bidder. This approach has been tested in communication bandwidth assignment problems, achieving satisfactory results, and we should study whether it could also fit the natural resources scenario we are dealing with. Also in relation with the strategic bidding, we should extend our model in order to deal with incentive compatibility issues such as taxes, penalties, discounts, etc. in order to complement our approach.

Unfortunately, as we have stated at the beginning, the current costs of natural resources are too cheap to use them as a unique motivation for industries to participate in the scheduling coordination mechanism presented here. What agents really want is to keep their current schedules, and scheduling coordinations offers this, with the possibility of getting a better social image as added value.

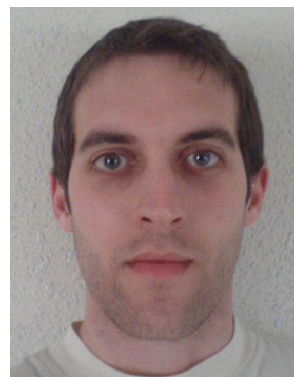
Acknowledgements We thank the Laboratory of Chemical and Environmental Engineering for having introduced us to the water treatment problem and for the help given during our research. This research project has been partially funded by DURSI AGAUR SGR 00296 (AEDS), and has had the support of the Commissioner for Universities and Research of the Department of Innovation, Universities and Company of Generalitat of Catalonia, the European Social Fund and the BR Grant Program from the Universitat de Girona.

References

- Arrow KJ, Sen AK, Suzumura K (eds) (2002) Handbook of social choice and welfare. North-Holland, Amsterdam
- Brucker P, Heitmann S, Hurink J (2003) Flow-shop problems with intermediate buffers. *OR Spectr* 25:549–574
- Chevaletre Y, Dunne P, Endriss U, Lang J, Lemaître M, Maudet N, Padget J, Phelps S, Rodríguez-Aguilar J, Sousa P (2006) Issues in multiagent resource allocation. *Informatica* 30:3–31
- Chevaletre Y, Endriss U, Estivie S, Maudet N (2007) Reaching envy-free states in distributed negotiation settings. In: Veloso M (ed) Proceedings of the 20th international joint conference on artificial intelligence (IJCAI-2007), Hyderabad, India, January 2007. pp 1239–1244. Poster paper
- Clearwater SH (ed) (1996) Market-based control: a paradigm for distributed resource allocation. World Scientific, River Edge. ISBN 981-02-2254-8
- Cox J, Durfee E (ed) (2003) Discovering and exploiting synergy between hierarchical planning agents. In: Proc of the second international joint conference on autonomous agents and multiagent systems, pp 281–288
- Cox JS, Durfee EH, Bartold T (2005) A distributed framework for solving the multiagent plan coordination problem. In: AAMAS '05: proceedings of the fourth international joint conference on autonomous agents and multiagent systems, New York, NY, USA, 2005. ACM Press, Cambridge, pp 821–827. ISBN 1-59593-093-0
- Cramton P, Shoham Y, Steinberg R (eds) (2006) Combinatorial auctions. MIT Press, Cambridge
- Dias MB, Stentz A (2000) A free market architecture for distributed control of a multirobot system. In: Proceedings of the 6th international conference on intelligent autonomous systems, pp 115–122
- Endriss U, Maudet N, Sadri F, Toni F (2003) Resource allocation in egalitarian agent societies. In: Herzig A, Chaib-draa B, Mathieu P (eds) *Secondes journées francophones sur les modèles formels d'interaction (MFI-2003)*. Cépaduès-Éditions, Paris, pp 101–110
- Gerkey BP, Mataric MJ (2002) Sold!: Auction methods for multi-robot coordination. *IEEE Trans Robot Automat* 18(5):758–768
- GLPK. GLPK (GNU Linear Programming Kit), <http://www.gnu.org/software/glpk/>
- Jeppsson U, Rosen C, Alex J, Copp J, Gernaey K, Pons M-N, Vanroellegem PA (2006) Towards a benchmark simulation model for plant-wide control strategy performance evaluation of wwtps. *Water Sci Technol* 53(1):287–295
- Kalagnanam J, Parkes D (2005) Auctions, bidding, and exchange design. In: *Handbook of supply chain analysis in the e-business era*. Kluwer Academic Publishers, Dordrecht
- Kelly T (2005) Generalized knapsack solvers for multi-unit combinatorial auctions: analysis and application to computational resource allocation. *LNAI* 3435:73–86
- Krauss S (2001) Strategic negotiation in multiagent environments. MIT Press, Cambridge
- Lee J-S, Szymanski B (2006) Auctions as a dynamic pricing mechanism for e-services. In: *Service enterprise integration*. Kluwer, Dordrecht, pp 131–156
- Modi P, Shen W, Tambe M, Yokoo M (2005) Adopt: Asynchronous distributed constraint optimization with quality guarantees. *Artif Intell J* 161:149–180
- Muñoz V, Murillo J (2008) CABRO: Winner determination algorithm for single-unit combinatorial auctions. In: *Artificial intelligence research and development (Proceedings of CCIA 2008)*. IOS Press, Utrecht, pp 303–312
- Payne T, David E, Jennings NR, Sharifi M (2006) Auction mechanisms for efficient advertisement selection on public displays. In: Brewka G, Coradeschi S, Perini A, Traverso P (eds) *ECAI*. IOS Press, Utrecht, pp 285–289
- Rendón-Sallard T, Sánchez-Marré M, Devesa F, Poch M (2006) Simulating scenarios for decision-making in river basin systems through a multi-agent system. In: Polit M, Talbert T, López B, Meléndez J (eds) *Proceedings of CCIA 2006*
- REPASt. Repast agent simulation toolkit, <http://repast.sourceforge.net>
- Salido MA, Barber F (2006) Distributed csps by graph partitioning. *Appl Math Comput* 183:491–498
- Tonino H, Bos A, Weerdt MD, Witteveen C (2002) Plan coordination by revision in collective agent-based systems. *Artif Intell* 142(2):121–145
- Wellman MP (1993) A market-oriented programming environment and its application to distributed multicommodity flow problems. *J Artif Intell Res* 1:1–23
- Zweben M, Fox M (1994) Intelligent scheduling. Morgan-Kaufmann, San Mateo



Javier Murillo received the MSc degree in Automatic Control and Computer Engineering in 2007. He is currently a PhD student in the Department of Electrical, Electronics and Automatic Engineering at University of Girona. His current research includes optimization, constraint satisfaction problems, combinatorial auctions, multi-agent systems and planning and scheduling.



Víctor Muñoz received the MSc degree in Automatic Control and Computer Engineering in 2008. He is currently a PhD student in the Department of Electrical, Electronics and Automatic Engineering at University of Girona. His current research includes optimization, robustness, constraint satisfaction problems, combinatorial auctions and complexity analysis.



Dídac Busquets is a research scientist in the University of Girona's Department of Electrical, Electronics and Automatic Engineering, where he's applying market mechanisms to task allocation and multiagent coordination. Busquets received his PhD in computer science from the Technical University of Catalonia in 2003. After doing a postdoctoral stay at the Robotics Institute of Carnegie Mellon University, he joined the University of Girona in 2006. His main research interest are multiagent systems, auction mechanisms and robotics.



Beatriz López is an associate professor in the University of Girona's Department of Electrical, Electronics and Automatic Engineering. Her research interests include multiagent systems, planning and scheduling, and case-based reasoning. López received her PhD in computer science from the Technical University of Catalonia. She's a member of the Catalan Association for Artificial Intelligence.